



**Tribhuwan University
Institute of Engineering
Pulchowk Campus**

Project Proposal

On

LoadBalancer Written in Go

Submitted By:

Prabesh Bastola (PUL080BCT052)

Sachin K.C. (PUL080BCT070)

Satyam Lamsal (PUL080BCT078)

Submitted To:

Department of Electronics and Computer Engineering

Subject: Network and System Programming

Instructor: Prabin Dhakal

Date: July 12, 2026

Abstract

GoBalancer GoBalancer is a production-grade load balancer implemented in Go that distributes client traffic across a pool of backend servers. It supports two operating modes: Layer 4 (TCP), which acts as a transparent byte-stream proxy compatible with any TCP-based protocol, and Layer 7 (HTTP), which functions as a reverse proxy capable of path and header aware request routing.

The system provides a unified load-balancing interface with four routing algorithms: Round Robin, Smooth Weighted Round Robin, Least Connections, and Consistent Hashing with virtual nodes. To improve reliability and operational resilience, it incorporates active and passive health checking, TLS termination, token-bucket rate limiting, zero-downtime configuration reloads, and comprehensive observability through Prometheus and Grafana.

The project is evaluated against five measurable objectives: preventing goroutine leaks under fault conditions, ensuring zero loss of in-flight requests during graceful shutdowns and configuration reloads, achieving sub-second failover when a backend becomes unavailable under load, providing real-time operational visibility through metrics and dashboards, and publishing a reproducible benchmarking report. Every major architectural decision is accompanied by a corresponding validation strategy, ensuring that implementation choices are supported by measurable evidence rather than assumptions.

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Problem Statement	4
1.3	Design Philosophy	4
1.4	Assumptions and Constraints	4
2	Background: Layer 4 vs. Layer 7	6
3	Goals and Non-Goals	7
3.1	Goals	7
3.2	Non-Goals	7
4	Feature Scope	8
4.1	Core Features	8
4.2	Load Balancing Algorithms	8
4.2.1	Round Robin	8
4.2.2	Smooth Weighted Round Robin	8
4.2.3	Least Connections	8
4.2.4	Consistent Hashing	9
4.3	Stretch Feature	9
5	The Health-Check Feedback Loop	10
6	Concurrency Model	11
6.1	Goroutine-per-Connection	11
6.2	Shared State and Synchronization	11
6.3	Goroutine Lifecycle and Leak Prevention	11
6.4	Backend Connection Handling	12
6.5	Backpressure and Resource Bounds	12
7	Architecture	14
7.1	System Overview	14
7.2	Main Components	14
7.3	Request Flow	14

8	Configuration	16
8.1	Configuration File	16
8.2	Configuration Validation	16
8.3	Hot Reload	17
8.4	TLS Support	17
9	Observability	18
9.1	Monitoring Metrics	18
9.2	Dashboard	18
9.3	Logging	18
10	Testing Strategy	19
10.1	Unit Testing	19
10.2	Integration Testing	19
10.3	Performance and Stress Testing	19
10.4	Chaos Testing	19
11	Benchmarking Plan	20
11.1	Benchmark Experiments	20
11.2	Methodology	20
12	Implementation Plan	21
13	Risks and Mitigations	22
14	Deliverables	22
	References	23

1 Introduction

1.1 Motivation

This project integrates many fundamental concepts in networking and concurrent programming into a single application. Building a production-style load balancer requires handling multiple client connections, managing shared resources safely, detecting server failures, and maintaining high availability under changing conditions.

This project focuses on following core areas of systems programming:

- **Socket programming** — accepting client connections, communicating with backend servers, and forwarding network traffic.
- **Concurrency** — handling thousands of simultaneous connections using Go's goroutines while safely managing shared resources such as backend health, configuration, and connection counts.
- **Fault tolerance** — automatically detecting failed backend servers, removing them from service, and adding them back when they recover.
- **Protocol awareness** — understanding the differences between Layer 4 (TCP) and Layer 7 (HTTP) load balancing, and the routing decisions that are possible at each layer.
- **System operations** — supporting configuration reloads, graceful shutdown, health monitoring, and performance metrics so the system can be operated reliably.

1.2 Problem Statement

Modern applications often use multiple backend servers to improve performance and reliability. Without a load balancer, client requests cannot be distributed efficiently, and the failure of a single backend may reduce the availability of the entire service.

The goal of GoBalancer is to distribute incoming client connections across multiple backend servers while maintaining high availability, efficient resource utilization, and fault tolerance. The system should continue operating correctly when backend servers fail, configuration changes are applied, or the load balancer itself is shut down.

1.3 Design Philosophy

The project follows three main design principles:

1. **Quality over quantity.** The focus is on implementing a smaller set of features correctly rather than adding many advanced features with limited testing.
2. **Everything should be measurable.** Important design decisions will be supported by benchmarks, monitoring data, and testing rather than assumptions.
3. **Incremental development.** The project will be built in small phases, with every phase producing a working and testable version of the system.

1.4 Assumptions and Constraints

The project is developed under the following assumptions and constraints:

- The development team consists of three members.

- Go 1.22 or newer will be used as the primary programming language.
- Linux is the primary development and testing platform.
- Third-party libraries will be kept to a minimum. The project primarily relies on Go's standard library together with a small number of external packages:
 - `golang.org/x/time/rate` for rate limiting;
 - `prometheus/client_golang` for metrics collection;
 - `gopkg.in/yaml.v3` for configuration parsing;
 - `fsnotify` (optional) for automatic configuration reload.
- Performance testing and benchmarking will be performed on a single documented machine using Docker Compose to run backend servers and the monitoring stack.

2 Background: Layer 4 vs. Layer 7

Load balancers can operate at different layers of the network stack. GoBalancer supports both Layer 4 (TCP) and Layer 7 (HTTP) load balancing. Although both distribute client traffic, they work with different information and therefore provide different capabilities.

Layer 4 operates at the transport layer and forwards TCP connections without inspecting the application data. This makes it suitable for any TCP-based protocol and provides high performance with minimal processing overhead.

Layer 7 operates at the application layer and understands HTTP requests. Because it can inspect request information such as URLs, headers, and cookies, it supports more advanced routing decisions but requires additional processing.

Aspect	L4 (TCP)	L7 (HTTP)
Visible Information	Source and destination IP addresses and ports	HTTP method, URL, headers, cookies, and request information
Routing	One backend is selected for each TCP connection	A backend can be selected for every HTTP request
Supported Protocols	Any TCP-based protocol	HTTP and HTTPS only
Load Balancing Key	Usually client IP address	Client IP, request path, header, or cookie
Performance	Minimal overhead because traffic is forwarded directly	Higher overhead because HTTP requests must be parsed
Failure Detection	Connection failures only	Connection failures, HTTP status codes, and request latency
TLS Handling	Encrypted traffic can be forwarded directly	HTTPS connections can be terminated at the load balancer

Table 1: Comparison of Layer 4 and Layer 7 load balancing.

Supporting both operating modes allows GoBalancer to demonstrate the trade-offs between performance and flexibility. Layer 4 provides efficient TCP forwarding with very little processing, while Layer 7 offers more intelligent request routing by understanding HTTP traffic. These differences will be evaluated through benchmarking in the final project.

Design Inspiration. The algorithms and techniques used in GoBalancer are based on ideas from widely used production load balancers such as NGINX, Envoy, and HAProxy. Features such as Smooth Weighted Round Robin, Consistent Hashing, and health checking follow well-established approaches instead of introducing new algorithms. Advanced features that are outside the scope of this project are listed in Section 3.2.

3 Goals and Non-Goals

3.1 Goals

ID	Goal	Acceptance criterion
G1	Correct concurrency	Zero goroutine leaks after a 5-minute chaos test: goroutine count returns to baseline, verified by <code>runtime.NumGoroutine()</code> sampling and diffed <code>pprof</code> goroutine profiles.
G2	Zero dropped in-flight requests	<code>SIGTERM</code> (shutdown) and <code>SIGHUP</code> (reload) issued under sustained load complete all already-accepted requests; verified by client-side error accounting in the load generator.
G3	Fast failover	When a backend is killed under load, the p99 client-visible disruption window is under 1 second.
G4	Observability	Prometheus metrics exposed and a Grafana dashboard committed to the repository; a backend kill and its recovery are visible on the dashboard within one probe interval.
G5	Published performance characterization	A benchmark report with throughput, latency percentiles (p50/p95/p99) against a direct-to-backend baseline, concurrency-scaling data, and annotated failover timelines — all reproducible via <code>make bench</code> .

Table 2: Project goals. G1–G3 are correctness goals and are never cut under schedule pressure (buffer policy, Section 12).

3.2 Non-Goals

The following features are intentionally excluded from this project so that development can focus on building a reliable and well-tested load balancer.

- **HTTP/2 and gRPC support.** These protocols are much more complex than HTTP/1.1 and require additional features such as stream multiplexing and flow control. Supporting them would significantly increase the project’s scope without contributing to its main learning objectives.
- **Service discovery.** Instead of integrating tools like Consul or etcd, backend servers will be listed in a YAML configuration file. The file can be reloaded without restarting the load balancer, providing a simpler way to manage backend changes.
- **Multiple load balancers (high availability).** This project focuses on a single load balancer. Running multiple load balancers introduces distributed-system challenges that are beyond the scope of this project.
- **Distributed rate limiting.** Rate limiting will be performed only within a single load balancer instance. Sharing rate limits across multiple machines is outside the scope of this project.

4 Feature Scope

4.1 Core Features

Feature	Description
L4 TCP Mode	Forwards TCP traffic transparently between clients and backend servers, making it suitable for any TCP-based application.
L7 HTTP Mode	Acts as an HTTP reverse proxy with support for path-based and header-based request routing.
Load Balancing	Supports Round Robin, Smooth Weighted Round Robin, Least Connections, and Consistent Hashing through a common interface.
Health Checking	Uses active health probes and passive failure detection to automatically remove unhealthy backends and restore them when they recover.
Graceful Shutdown	Stops accepting new requests while allowing existing requests to complete before shutting down.
TLS Termination	Handles HTTPS connections and supports certificate reloading without restarting the server.
Timeout Management	Applies connection and request timeouts throughout the system to prevent stalled or hanging connections.
Configuration Reload	Reloads backend configuration from a YAML file without interrupting active client connections.
Observability	Provides Prometheus metrics and Grafana dashboards for monitoring performance and backend health.
Rate Limiting	Uses token-bucket rate limiting globally and per client IP to protect backend servers from excessive traffic.

Table 3: Core features of GoBalancer.

4.2 Load Balancing Algorithms

GoBalancer supports four load balancing algorithms. Each algorithm can be selected through the configuration file and operates only on healthy backend servers.

4.2.1 Round Robin

Requests are distributed evenly across all healthy backend servers in a cyclic order. This approach is simple, efficient, and works well when all backends have similar capacity.

4.2.2 Smooth Weighted Round Robin

Backends can be assigned different weights based on their processing capacity. Servers with higher weights receive more requests while the smooth scheduling algorithm prevents long bursts of consecutive requests to the same backend.

4.2.3 Least Connections

Each incoming request is sent to the healthy backend currently serving the fewest active connections. This helps balance workloads when request processing times vary.

4.2.4 Consistent Hashing

Consistent Hashing routes requests based on a hash of a client-specific key, such as the client IP address or an HTTP header. Virtual nodes are used to improve load distribution across the hash ring. When backend servers are added or removed, only a small portion of clients are remapped, making this algorithm suitable for sticky sessions and cache affinity.

4.3 Stretch Feature

If all core features are completed on schedule, an additional feature will be implemented:

SNI-based routing in L4 mode. The load balancer will inspect the TLS ClientHello message to determine the requested hostname (SNI) without decrypting the traffic. Requests can then be forwarded to the appropriate backend while preserving end-to-end TLS encryption.

5 The Health-Check Feedback Loop

GoBalancer combines **passive** and **active** health checking to detect backend failures and recover them safely. Rather than treating these as separate features, they work together using the principle of “**fail fast, recover carefully.**”

- **Passive health checks.** Every client request is monitored. If a backend fails several consecutive requests (default: 3), it is immediately marked as unhealthy and removed from the pool. This allows the load balancer to react much faster than waiting for the next scheduled health check.
- **Active health checks.** A background process periodically sends health-check requests to unhealthy backends. After a cooldown period and two consecutive successful health checks (default), the backend is added back to the pool. This prevents unstable servers from being reintroduced too quickly.

This strategy reduces the number of client-visible failures while also avoiding frequent removal and re-addition of unstable backends.

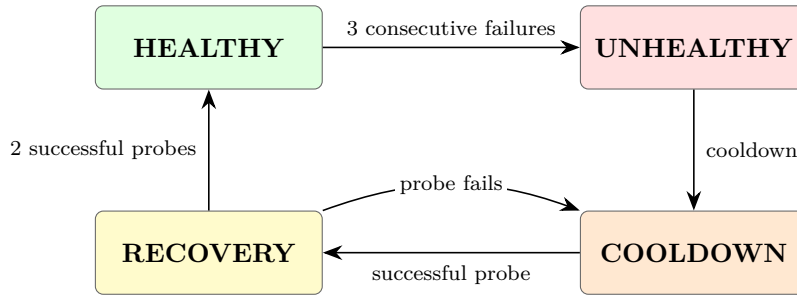


Figure 1: Backend health state transitions. Failed servers are removed quickly, while recovered servers must pass multiple health checks before they receive traffic again.

Each backend maintains its own health state. State changes are recorded and exposed through Prometheus metrics, allowing backend failures and recoveries to be visualized in Grafana dashboards.

Parameter	Default	Purpose	Description
Failure threshold	3	Passive	Consecutive failures before marking a backend unhealthy
Cooldown period	10s	Recovery	Minimum waiting time before retrying an unhealthy backend
Success threshold	2	Active	Consecutive successful probes required before recovery
Probe interval	2s	Active	Time between health checks
Probe timeout	500ms	Active	Maximum time to wait for a health-check response
Health endpoint	/health	Active	HTTP endpoint used for health checks

Table 4: Default health-check configuration. All values can be changed through the configuration file.

With these default settings, failed backends can usually be detected in well under one second during heavy traffic, allowing the load balancer to quickly redirect requests to healthy servers. The effectiveness of this approach will be evaluated through benchmarking and failover testing.

6 Concurrency Model

The concurrency model is the core of GoBalancer. This section explains how concurrent connections are handled, how shared data is synchronized, and how goroutine leaks and resource exhaustion are prevented. Every design decision is supported by testing and benchmarking.

6.1 Goroutine-per-Connection

Each client connection is handled by dedicated goroutines. In L4 (TCP) mode, two goroutines are created for every connection: one copies data from the client to the backend, while the other copies data in the opposite direction.

- Go's runtime schedules many goroutines onto a smaller number of operating system threads using the **netpoller** (**epoll** on Linux) [5]. When a goroutine waits for network I/O, only that goroutine is paused, allowing the thread to continue executing other goroutines. This makes the goroutine-per-connection model efficient even with thousands of concurrent connections.
- A new goroutine starts with a small stack (approximately 2–8 KB), making it practical to support thousands of concurrent connections. Although a manually written **epoll** event loop could reduce memory usage slightly, it would significantly increase code complexity and make connection handling harder to maintain.
- **Verification:** Benchmark results will record the number of goroutines and memory usage (RSS) while handling 100, 1 000, 5 000, and 10 000 concurrent connections (Section 11).

6.2 Shared State and Synchronization

Different synchronization techniques are used depending on how often a piece of shared data is read or modified. Frequently accessed counters use atomic operations to avoid locks, while mutexes are used only when shared state must be updated safely.

6.3 Goroutine Lifecycle and Leak Prevention

Every goroutine created by the load balancer has a clearly defined exit condition. Goroutines terminate when a connection closes, a backend fails, or the associated context is cancelled. This prevents goroutine leaks during long-running operation.

The most important case is TCP proxying. Two goroutines use `io.Copy()` to transfer data between the client and backend. If one direction finishes first, the connection cannot simply be closed, because the other direction may still be sending data. To handle this correctly:

1. When one `io.Copy()` finishes, call `CloseWrite()` on the destination connection. This sends an EOF while allowing communication to continue in the opposite direction.
2. Apply a deadline to the remaining connection so that a stalled peer cannot keep the goroutines alive indefinitely.
3. Once both directions finish (or time out), close both connections completely and update the active connection count.

Verification: Chaos testing (Section ??) intentionally terminates backend servers while traffic is flowing. During these tests, `runtime.NumGoroutine()` is monitored continuously and `pprof` goroutine profiles are compared before and after the test. The goroutine count must always

Shared State	Synchronization	Reason
Configuration / Backend Pool	<code>atomic.Pointer</code> swap	Configuration is replaced atomically during reloads so readers never need to wait for a lock.
Round-Robin Counter	<code>atomic.Uint64</code>	Updated for every request, so lock-free access provides the best performance.
Per-backend Connection Count	<code>atomic.Int64</code>	Updated whenever connections are opened or closed, and also used for Least Connections and monitoring metrics.
Backend Health State	Atomic state with per-backend mutex	Health status is read frequently but changes only occasionally, making this combination efficient.
Consistent Hash Ring	Immutable structure with atomic pointer swap	The hash ring is rebuilt only when backend membership changes, allowing normal request routing to remain lock-free.
Smooth Weighted Round Robin	Mutex	Selecting the next backend requires updating shared weights, so a mutex is required. Benchmarking will verify that this does not become a performance bottleneck.

Table 5: Synchronization strategy for shared data.

return to its original baseline. These checks are included in continuous integration from Phase P1 onward.

6.4 Backend Connection Handling

Backend connections are managed differently depending on the operating mode.

- **L4 (TCP):** Every client connection creates a separate backend connection because a transparent TCP proxy cannot share connections between clients. Connection timeouts also provide input to the passive health checker.
- **L7 (HTTP):** Backend connections are reused through Go’s `http.Transport`, reducing the overhead of repeatedly creating TCP connections. The maximum number of idle connections can be configured.

Instead of using Go’s built-in `httputil.ReverseProxy`, GoBalancer implements a lightweight reverse proxy directly on top of `http.Transport`. This provides greater control over request handling, including header processing, request forwarding, streaming response bodies, and request cancellation. The implementation will be validated by comparing its behaviour with the standard library proxy. If the custom proxy cannot be completed within the project timeline, the standard library implementation will be used as a fallback.

6.5 Backpressure and Resource Bounds

GoBalancer places limits on resource usage to prevent overload and maintain stable performance.

- **Fail fast.** When the maximum number of connections or the configured rate limit is reached, new requests are rejected immediately instead of waiting in an unbounded queue. L4 connections are reset, while L7 clients receive an HTTP 503 response with a `Retry-After` header.

- **Timeouts prevent resource exhaustion.** Every backend operation is protected by connection and I/O timeouts. Slow or unresponsive backends therefore fail quickly, allowing the passive health checker to remove them before blocked goroutines accumulate.
- **Bounded memory usage.** Auxiliary data structures such as the per-client rate limiter use an LRU cache with a fixed size, and monitoring metrics use a limited set of labels to avoid excessive memory consumption.

7 Architecture

7.1 System Overview

Figure 2 shows the overall architecture of GoBalancer. Client requests first pass through the listener and rate limiter before being routed to a healthy backend using the selected load balancing algorithm. Supporting components such as the health manager, configuration manager, and metrics system operate alongside the main request path to ensure the system remains reliable and easy to monitor.

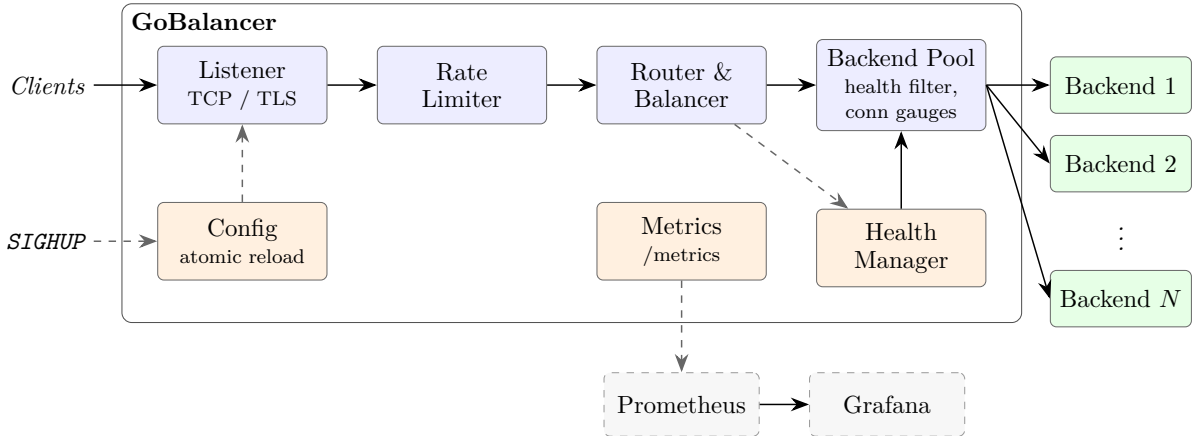


Figure 2: High-level architecture of GoBalancer. Solid arrows represent the flow of client requests, while dashed arrows represent configuration updates and monitoring.

7.2 Main Components

The load balancer is divided into several independent components, each responsible for a specific task.

- **Listener** accepts incoming TCP or TLS connections.
- **Rate Limiter** protects the system by limiting the number of requests globally and per client.
- **Router and Balancer** selects the appropriate backend pool and chooses a healthy backend using the configured load balancing algorithm.
- **Backend Pool** maintains the list of available backend servers together with their health status and connection counts.
- **Health Manager** continuously monitors backend servers, removes failed servers from service, and adds them back after successful health checks.
- **Configuration Manager** loads the YAML configuration and supports hot reloading without interrupting existing connections.
- **Metrics System** exports Prometheus metrics that are visualized using Grafana dashboards.

7.3 Request Flow

Layer 7 (HTTP)

1. A client connects to the load balancer.

2. The request passes through the rate limiter.
3. The HTTP request is examined to determine the target backend pool.
4. Unhealthy backends are removed from consideration.
5. The selected load balancing algorithm chooses one backend.
6. The request is forwarded to that backend.
7. The response is streamed back to the client while metrics and health information are updated.

Layer 4 (TCP)

The overall process is similar to Layer 7, except that TCP traffic is forwarded without inspecting the application data. Backend selection is typically based on the client IP, and traffic is copied directly between the client and backend until either side closes the connection.

8 Configuration

8.1 Configuration File

GoBalancer uses a single YAML configuration file to define how the load balancer operates. The configuration includes network settings, load balancing mode, backend servers, health checks, rate limits, TLS settings, and routing rules (for HTTP mode).

An example configuration is shown below.

```
mode: 17
listen: ":8443"

tls:
  cert: /etc/gobalancer/cert.pem
  key: /etc/gobalancer/key.pem

balancer: consistent_hash

timeouts:
  dial: 2s
  read: 30s
  write: 30s

health:
  active:
    interval: 2s
    timeout: 500ms
    rise: 2
  passive:
    fall: 3
    cooldown: 10s

rate_limit:
  global_rps: 5000
  per_client_rps: 100

routes:
  - match:
      path_prefix: /api/
      pool: api

pools:
  api:
    - addr: "10.0.0.1:8080"
      weight: 3
    - addr: "10.0.0.2:8080"
      weight: 1
```

The actual configuration may change during development, but it will follow this overall structure.

8.2 Configuration Validation

Before a configuration is applied, GoBalancer performs several validation checks, including:

- backend addresses must be valid and unique;

- backend weights must be positive;
- every routing rule must reference an existing backend pool;
- timeout values must be valid;
- TLS certificates and keys must be valid when TLS is enabled.

If any validation fails, the new configuration is rejected and the existing configuration continues running without interruption.

8.3 Hot Reload

GoBalancer supports configuration reload without restarting the application.

When a **SIGHUP** signal (or an optional file change event) is received, the new configuration is loaded and validated. If validation succeeds, it replaces the current configuration using an atomic update. Existing client connections continue using the old configuration until they finish, while new connections immediately use the updated configuration.

This design allows configuration changes with zero dropped connections, which will be verified during system testing.

8.4 TLS Support

GoBalancer supports TLS termination with a minimum version of TLS 1.2. Certificates and private keys are loaded from the configuration file and can be updated through the same hot reload mechanism. Existing TLS connections continue normally, while new connections use the updated certificate. Secure TLS settings are provided by Go's `crypto/tls` package instead of custom implementations.

9 Observability

9.1 Monitoring Metrics

GoBalancer will expose Prometheus metrics to monitor system performance, backend health, and resource usage. These metrics will help evaluate the system during development and benchmarking.

Metric	Type	Purpose
Request count	Counter	Tracks the total number of requests handled by each backend.
Active connections	Gauge	Shows the number of live connections for each backend.
Request latency	Histogram	Measures response time and calculates p50, p95, and p99 latency.
Backend health	Gauge	Indicates whether each backend is healthy or unhealthy.
Health transitions	Counter	Records when backends are removed from or returned to service.
Rate-limited requests	Counter	Tracks requests rejected by the rate limiter.
Configuration reloads	Counter	Counts successful and failed configuration reloads.
Go runtime metrics	Built-in	Monitors goroutines, memory usage, and garbage collection.

Table 6: Key Prometheus metrics collected by GoBalancer.

9.2 Dashboard

The collected metrics will be visualized using Grafana. The dashboard will include:

- request rate and response latency;
- backend health status and active connections;
- system resource usage, including goroutines and memory;
- rate-limited requests and configuration reload events.

During the final demonstration, the dashboard will show how the load balancer responds when a backend server fails and later recovers.

9.3 Logging

GoBalancer will use structured logging to record important system events such as client connections, backend failures, health status changes, and configuration reloads. These logs will assist with debugging, testing, and performance analysis throughout the project.

10 Testing Strategy

Automated testing will be performed using Go's testing framework, together with the race detector (`go test -race`), static analysis (`go vet`), and a linter.

10.1 Unit Testing

Unit tests will verify the correctness of individual components in isolation.

- Load balancing algorithms (Round Robin, Weighted Round Robin, Least Connections, and Consistent Hashing).
- Health checking logic, including backend failure detection and recovery.
- TCP proxy functionality, including proper connection closing and bidirectional data transfer.
- Configuration parsing and validation.
- Rate limiting behavior.

10.2 Integration Testing

Integration tests will verify that multiple components work together correctly.

- End-to-end request forwarding in both Layer 4 (TCP) and Layer 7 (HTTP) modes.
- Graceful shutdown without interrupting existing client connections.
- Configuration hot reload while the system is handling traffic.
- Backend failure and automatic failover.
- Verification that monitoring metrics and health status are updated correctly.

10.3 Performance and Stress Testing

Performance testing will measure the throughput, latency, and resource usage of GoBalancer under different workloads. The results will be compared with direct backend communication to evaluate the overhead introduced by the load balancer.

Stress tests will also be performed by increasing the number of concurrent client connections to evaluate scalability and identify resource limitations.

10.4 Chaos Testing

A dedicated chaos test will simulate backend failures while the load balancer is serving live traffic. During these tests, backend servers will be stopped and restarted repeatedly to verify that:

- failed backends are detected automatically;
- client traffic is redirected to healthy backends;
- recovered backends are added back into service;
- no goroutine leaks or deadlocks occur;
- monitoring dashboards accurately reflect system state.

These tests directly validate the project's main goals of correctness, fault tolerance, and high availability.

11 Benchmarking Plan

The project will be benchmarked to evaluate its performance, scalability, and reliability under different workloads. The results will be compared with direct backend communication to measure the overhead introduced by the load balancer.

11.1 Benchmark Experiments

Experiment	Purpose
Baseline performance	Compare direct backend communication with the load balancer to measure additional latency and throughput overhead in both L4 and L7 modes.
Concurrency scaling	Measure latency, memory usage, and goroutine count while increasing the number of concurrent connections (100, 1k, 5k, and 10k).
Load-balancing algorithms	Compare Round Robin, Weighted Round Robin, Least Connections, and Consistent Hashing under different traffic patterns.
Consistent hashing	Measure how many client requests are remapped when a backend is added or removed.
Failover performance	Measure how quickly traffic recovers after a backend failure and verify that failover happens within the project target.

Table 7: Planned benchmark experiments.

11.2 Methodology

Benchmarks will be performed on a single documented machine using Docker Compose to run the load balancer and backend servers. Load will be generated using tools such as `wrk` and `vegeta`. Each experiment will be executed multiple times, and the average throughput, latency (p50, p95, and p99), memory usage, and goroutine count will be recorded. CPU, memory, and goroutine profiles will also be collected using `pprof`. All benchmark scripts will be included in the repository so the results can be reproduced using `make bench`.

12 Implementation Plan

Each phase ends in a working, demoable system.

Phase	Deliverable	Exit criterion
P0 Skeleton	Repository setup; CI (<code>go vet</code> , <code>-race</code> , <code>lint</code>); configuration parsing and validation; structured logging	<code>make test</code> passes in CI
P1 L4 Core	TCP proxy with half-close handling; Round Robin balancing; timeouts; graceful shutdown; goroutine leak detection	Load test passes; graceful shutdown works; no goroutine leaks
P2 Health Loop	Active health checks; passive failure detection; health state machine; backend filtering	Traffic automatically reroutes when a backend fails
P3 L7 + TLS	HTTP reverse proxy; path/header routing; TLS termination; proxy equivalence tests	Both L4 and L7 modes work correctly
P4 Algorithms	Weighted Round Robin; Least Connections; Consistent Hashing	All balancing algorithm tests pass
P5 Operations	Hot configuration reload; rate limiting; Prometheus metrics; Grafana dashboard; Docker Compose setup	Reload works without dropping requests; dashboard displays metrics
P6 Benchmark & Report	Benchmark suite; chaos testing; final report; recorded demonstration	All benchmark experiments are reproducible using <code>make bench</code>
Stretch	SNI-based routing in L4 mode (Section 4.3)	Implemented only if all core phases finish early

Table 8: Phased implementation plan.

Buffer policy. If the project falls behind schedule, additional balancing algorithms (Weighted Round Robin and Least Connections) and the stretch feature will be removed first. The core correctness goals (G1–G3), observability, and benchmarking will always remain part of the project.

13 Risks and Mitigations

Risk	Likelihood	Mitigation
Goroutine leaks in edge cases: half-close, mid-transfer backend kill, client abort during dial	High — <i>the classic bug in this design</i>	Leak check is a CI test from P1, not a final audit; every goroutine reviewed against an exit-path checklist; the chaos test exercises exactly these paths.
Hot-reload races (request observing mixed old/new config)	Medium	Immutable snapshots + single atomic swap make the race structurally impossible; -race in CI; dedicated reload-under-load test.
Hand-rolled L7 proxy correctness (hop-by-hop headers, streaming, upgrades)	Medium	Behavioral-equivalence test against <code>httputil.ReverseProxy</code> ; fallback at end of P3 to the stdlib proxy if needed.
Benchmark noise (thermal throttling, Docker overhead, coordinated omission)	Medium	Fixed backend response times; constant-rate load generator for latency [6]; environment documented; three runs with variance reported.
Scope creep	Medium	The non-goals list (Section 3.2) is contractual; the stretch feature is gated on P6 completion.
Schedule slip	Medium	Buffer policy of Section 12; every phase is independently demoable, so a slip truncates features, never correctness.

Table 9: Risk register.

14 Deliverables

1. Source repository with CI: unit + integration tests under **-race**, **go vet**, linting.
2. Docker Compose demo environment: GoBalancer, demo backends, Prometheus, Grafana (auto-provisioned dashboard).
3. Committed Grafana dashboard JSON.
4. Benchmark report (`docs/benchmarks.md`) with raw data, reproducible via `make bench`.
5. Chaos-test harness and archived results (goroutine profiles, error timelines).
6. Final written report defending the design decisions of Sections 5 and 6, plus a recorded failover demo.

References

- [1] Go standard library documentation: `net`, `net/http`, `net/http/httputil`, `crypto/tls`, `context`, `log/slog`, `hash/fnv`. <https://pkg.go.dev>
- [2] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine, and D. Lewin, “Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web,” in *Proc. 29th ACM Symposium on Theory of Computing (STOC)*, 1997.
- [3] nginx development team, upstream module: smooth weighted round-robin balancing (source commit notes), <https://github.com/nginx/nginx>.
- [4] Envoy Proxy documentation, “Outlier Detection”; HAProxy Configuration Manual, `observe`, `rise`, `fall` directives — prior art for the asymmetric evict/readmit policy.
- [5] The Go runtime netpoller (`runtime/netpoll_epoll.go` and associated design discussions) — basis for the goroutine-per-connection argument.
- [6] G. Tene, “How NOT to Measure Latency,” talk, 2015 — coordinated omission and constant-rate load generation.